CHAPTER I

INTRODUCTION AND OBJECTIVES

1.1. Introduction:

Serial robotic manipulators are foundational to modern automation. While high-precision industrial manipulators are well-studied, there is a growing interest in low-cost, 3D-printed, and open-source robotic arms, particularly as testbeds for advanced control concepts. This study focuses on a 4-DOF serial manipulator, as specified by the updated configuration (Base-Twist, Shoulder-Bend, Elbow-Revolute, EE-Roll). The physical context for this research is a prototype that is a 5-DOF + Gripper system (utilizing 6 servos). For the purpose of this rigorous analysis, we adhere to the specified 4-DOF (RRRR) kinematic chain. The baseline control logic, and the hardware constraints, such as the use of MG995/SG90 servos and an ESP8266 controller, are directly applicable.

The primary challenge in such systems is the "reality gap" between classical robotic control theory and the hardware's practical limitations. Classical model-based control strategies, such as Computed Torque Control (CTC), presume the ability to command joint torques directly. The hardware used here violates this assumption; hobbyist servos are position-controlled devices with internal, low-cost controllers. They suffer from significant unmodeled dynamics, including gear backlash, actuator deadband, and nonlinear friction. Compounding this, the ESP8266 controller, while powerful in connectivity, is not a real-time system. Its Wi-Fi stack and other processes introduce interrupts that cause significant jitter in software-generated Pulse-Width Modulation (PWM) signals, leading to erratic servo motion. Furthermore, the high combined inrush current of the actuators can overwhelm a standard 5V power supply, causing voltage sags (brownouts) that lead to system resets and instability.

Therefore, the central problem is to design an achievable trajectory and control strategy that mitigates these hardware deficiencies. This research posits that for such systems, optimizing the trajectory planner for high-order smoothness (e.g., minimum-jerk) is more critical than implementing a complex, infeasible feedback controller, as it prevents the excitation of the system's unstable, nonlinear dynamics.

1.2. Literature Review and Problem Statement

This study is guided by the overarching goal of transforming the 4-DOF manipulator from an imprecise, teleoperated device (as implied by the baseline control logic) into a platform capable of autonomous, optimized trajectory tracking. The specific technical objectives are:

a) Manipulator Modeling (Domain A): To develop a complete kinematic model of the 4-DOF (RRRR) serial manipulator using the Denavit-Hartenberg (D-H) parameterization. This includes deriving the forward kinematic (FK) transformation matrices, an analytical inverse kinematic (IK) solution for 3D position and tool roll, and the manipulator Jacobian for singularity analysis.
b) Baseline System Benchmarking (Domain D): To formally analyze the baseline system architecture, as documented in the baseline system, identifying the open-loop, non-real-time control logic and quantifying its limitations, including PWM jitter, power instability, and actuator nonlinearities.
c) Trajectory Planning Optimization (Domain B): To design, compare, and implement an optimal joint-space trajectory planner. This involves evaluating trapezoidal velocity profiles, cubic polynomials, and quintic (minimum-jerk) polynomials, with the objective of achieving C2 continuity (continuous acceleration) to minimize mechanical jerk and stabilize power consumption.
d) Control System Optimization (Domain C): To analyze the feasibility of advanced control strategies (e.g., CTC, SMC) against the hardware constraints. A hardware-aware, computationally efficient kinematic control law (e.g., PD with Velocity Feedforward) that is compatible with the ESP8266 and the servo's position-control interface will be designed.
e) Simulation and Performance Evaluation (Domain E): To develop a simulation environment (e.g., MATLAB/Simulink, Python) to conduct a comparative "Before vs. After" analysis, quantifying performance improvements using standard metrics (RMS error, max error, jerk).
f) Validation and Discussion (Domain F): To validate the simulation results on the physical hardware prototype. This involves implementing calibration procedures and comparing simulated vs. real-world performance, thereby quantifying the "reality gap".
g) Analysis and Recommendations (Domain G): To synthesize all findings, discuss trade-offs, identify residual error sources, and provide recommendations to address the identified research gaps in low-cost manipulator control.

CHAPTER II

METHODOLOGY FOR INCREMENTAL OPTIMIZATION

2.1. Baseline System Architecture:

The Baseline System, Documented in The Baseline System, Serves as the "Before" Case for Optimization.

Hardware Components (Domain D):

a) Controller: Visual inspection of the wired prototype confirms the use of an ESP8266-based board (a NodeMCU variant). This is a 32-bit microcontroller running a non-real-time, Arduino-based framework. While a conceptual wiring diagram shows an Arduino UNO, the physical build and baseline control code clearly indicate an ESP8266.
b) Actuators: The 3D-printed arm is wired with multiple servos. The baseline code confirms control for six servos, consistent with a 5-DOF kinematic chain plus a gripper. The actuators are hobbyist-grade, such as TowerPro MG995s and SG90s.
    i) MG995: Provides high stall torque but suffers from a significant deadband width (approximately 5 microseconds, which is a 'dead zone' where small commands are ignored), high stall current (approximately 1200 mA), and notable gear backlash (a 'looseness' or 'slop' in the gears).
    ii) SG90: A low-cost, 9g micro-servo with plastic gears, low torque (approximately 1.8 kg-cm), and susceptibility to backlash.
c) Power: A 5V power supply, as implied by the conceptual wiring schematic and typical ESP8266 projects.

Baseline Software and Control Logic:

a) Libraries: The system relies on ESP8266WiFi.h, BlynkSimpleEsp8266.h, and the standard Servo.h library.
b) Logic: The control mechanism is remote teleoperation. Six BLYNK_WRITE(Vn) functions asynchronously receive integer angle values from a mobile application. These values are passed directly to the servo.write(angle) command.

Analysis of Baseline Limitations: This architecture is critically flawed for trajectory tracking:

a) No Trajectory: The system does not "plan" motion; it teleports between discrete setpoints. This high-jerk "bang-bang" motion (instant acceleration and deceleration) excites the system's worst-case dynamics, maximizing stress, current draw, and the effect of gear backlash.
b) Non-Real-Time Execution: The loop() is dominated by Blynk.run(), which manages the Wi-Fi stack. Wi-Fi interrupts frequently block the software-timed Servo.h library interrupts, causing inconsistent pulse widths and severe, visible servo jitter (shaking).
c) Power System Instability: This is a critical failure point. The servos (especially multiple MG995s) can draw a combined stall/inrush current of many amps. A standard 5V USB or breadboard power supply cannot meet this demand, resulting in voltage sag (brownout) that resets the ESP8266.
d) Open-Loop Control: The servo.write(angle) command is an open-loop setpoint instruction from the ESP8266's perspective. There is no feedback to confirm if the servo reached the position.

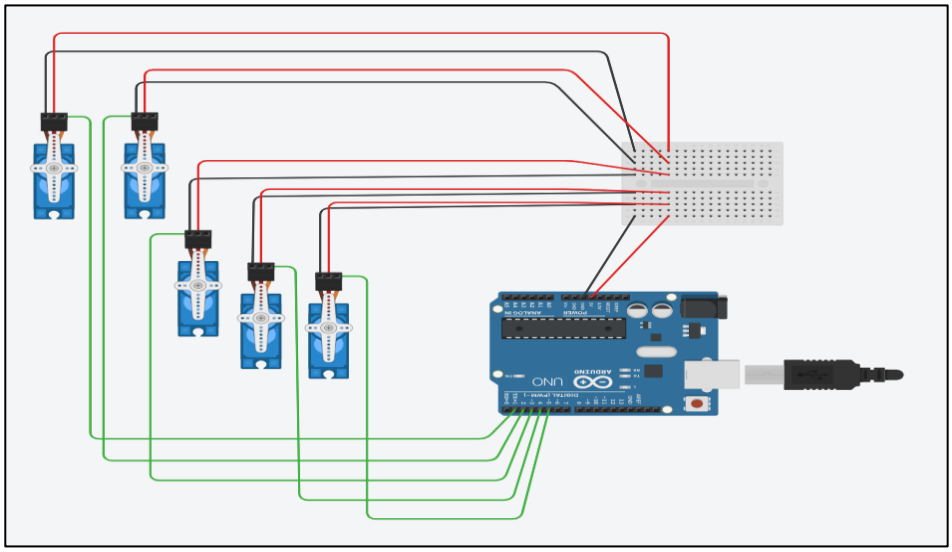

Figure 1: Schematic of Servo Motor Interfacing with Arduino Uno.

2.2. System Evaluation and Benchmarking:

To optimize the system, a formal mathematical model (Domain A) is required for the specified 4-DOF RRRR (Base, Shoulder, Elbow, Roll) configuration. The Denavit-Hartenberg (D-H) convention is the standard method.

Figure 2: A 4-link active serial manipulator. Frame 0 is at the base. Joint 1 (Base) rotates. Joint 2 (Shoulder) pitches. Joint 3 (Elbow) pitches. Joint 4 (Wrist Roll) rolls. Frame 4, the end-effector frame, is attached.

Table 2.2.1: Denavit-Hartenberg Parameters for the 5-DOF Manipulator:

Joint (i)	Link Twist (α_{i-1})	Link Length (a_{i-1})	Link Offset (d_i)	Joint Angle (θ_i)
1	0	0	d_1 (base height)	θ_1
2	π/2	a_1 (link 1)	0	θ_2
3	0	a_2 (link 2)	0	θ_3
4	π/2	0	d_4 (wrist/EE)	θ_4

Forward Kinematics (FK): The homogeneous transformation matrix $\boldsymbol{A_i}$ relating frame ***i*** to frame ***i-1*** is given by:

$$\boldsymbol{A_{i-1}^{i} = Rot_Z(\theta_I).Trans_z(d_i).Trans_x(a_{i-1}).Rot_x(a_{i-1})}$$

The complete FK, mapping the joint vector $q = [\theta_1, \theta_2, \theta_3, \theta_4]^T$ to the end-effector pose T_0^4 (a 4x4 matrix), is the product of these transformations:

$$\boldsymbol{T_0^4 = A_0^1(\theta_1)A_1^2(\theta_2)A_2^3(\theta_3)A_3^4(\theta_4)}$$

Inverse Kinematics (IK): The IK problem involves finding q for a desired end-effector pose. A 4-DOF manipulator cannot achieve an arbitrary 6-DOF pose (3 position, 3 orientation). However, for the specified RRRR configuration, it can uniquely solve the task of 3D position *(x, y, z)* and end-effector roll. A closed-form analytical or geometric solution is preferred for its speed and reliability. The geometric approach is common for this structure:

a) **Solve for $\boldsymbol{\theta_1}$:** The position of the wrist centre is projected onto the X_0-Y_0 (base) plane. θ_1 is found atan2.
b) **Solve for $\boldsymbol{\theta_2}$ *and* $\boldsymbol{\theta_3}$:** with θ_1 known, the problem reduces to a 2D, 2-link planar arm in the vertical plane. θ_2 *and* θ_3 are found using the law of cosines, yielding two solutions ("elbow up" and "elbow down").
c) **Solve for $\boldsymbol{\theta_4}$:** It is then calculated to satisfy the desired tool roll orientation relative to the arm's configuration. A complete closed-form solution for 4-DOF manipulators is a well-documented problem.

Jacobian and Singularity Analysis: The manipulator Jacobian *J(q)* is a 6x4 matrix that maps joint velocities $\dot{\boldsymbol{q}}$ **(**a 4x1 vector) to end-effector linear and angular velocities $\boldsymbol{v}$ (a 6x1 vector).

$$\boldsymbol{V} = [\dot{\boldsymbol{x}} \quad \dot{\boldsymbol{y}} \quad \dot{\boldsymbol{z}} \quad \boldsymbol{\omega_x} \quad \boldsymbol{\omega_y} \quad \boldsymbol{\omega_z}]^{\boldsymbol{T}}$$

Singularities occur when the Jacobian *J(q)* loses rank. At these configurations, the manipulator loses one or more degrees of freedom. Common singularities for this RRRR arm include: Shoulder and Elbow Singularity.

2.3. Optimizing the Control System:

The most significant hardware constraint (Domain D) is that hobbyist servos are position-controlled, not torque-controlled. This makes standard dynamic control laws infeasible.

Analysis of Infeasible Control Strategies (Domain C):

a) PD + Gravity Compensation: The control law $\tau = K_p e + K_d \dot{e} + g(q)$ computes a vector of torques. This cannot be sent to the servos.
b) Computed Torque Control (CTC): The law $\tau = M(q)\left(\ddot{q}_d + K_v \dot{e} + K_p e\right) + C(q, \dot{q}) + g(q)$ is the gold standard for tracking, as it linearizes the manipulator's dynamics. However, it is doubly infeasible: it outputs torque τ, and its real-time computation of the mass matrix $M(q)$ and Coriolis vector $C(q, \dot{q})$ is too demanding for an ESP8266.

c) Sliding Mode Control (SMC): While robust to the unmodeled dynamics and parameter uncertainties that plague this arm, SMC also computes a torque-based control signal.

Table 2.3.1: Comparison of Control Strategies and Hardware Feasibility:

Control Strategy	Control Variable	Model-Based (Dynamics)	Computational Load	Feasible on Hobby Servo?
Baseline	Position	No	Very Low	Yes (ineffective)
PD + Gravity	Torque	Partial (Gravity)	Low-Medium	No
Computed Torque	Torque	Full (M, C, G)	High	No
Sliding Mode	Torque	Partial/Full	Medium-High	No
Optimized Kinematic + FF	**Position**	**No (Kinematic)**	**Very Low**	**Yes (Optimized)**

Selected Optimized Strategy: Kinematic Control with Velocity Feedforward

The optimization must pivot from dynamic control (calculating torque) to kinematic control (calculating a smarter position setpoint). We are not replacing the servo's internal PID; we are pre-compensating its input.

The optimized control law computes the commanded angle $q_{cmd}(t)$ to send to the servo at each time step Δt (e.g., 20 ms). The trajectory planner (Sec 2.4) provides the desired position q_d(t) and desired velocity $\dot{q}_d(t)$. A simple velocity feedforward (FF) term is added:

$$\boldsymbol{q_{cmd}(t) = q_d(t) + K_{ff} * \dot{q}_d(t)}$$

Where K_{ff} is a feedforward gain. This "leads" the trajectory, telling the servo to move to a position slightly ahead of the desired path. This effectively anticipates and compensates for the lag of the servo's internal controller, reducing tracking error. This entire calculation is computationally trivial for the ESP8266.

2.4. Simplifying and Enhancing Trajectory Planning

With a control law limited to kinematic feedforward, the quality of the trajectory (Domain B) becomes the single most important factor for optimization. The trajectory must be smooth to avoid exciting the hardware's worst-case behaviors (backlash, power sag).

Baseline: Trapezoidal Velocity Profile, A simple "point-to-point" move is often done with linear interpolation, which results in a trapezoidal velocity profile.

a) Position: C1 continuous (continuous velocity).
b) Velocity: C0 continuous (discontinuous acceleration).
c) Acceleration: Discontinuous step functions.
d) Jerk: The derivative of acceleration. It is infinite at the points of acceleration change. This infinite jerk commands an instantaneous change in torque, causing a massive current spike from the servo. This is a primary cause of power instability.

Option 1: Cubic Polynomials

A cubic polynomial trajectory q(t) = $a_0 + a_1 t + a_2 t^2 + a_3 t^3$ can be solved using four constraints: initial/final position (q_0, q_f) and initial/final velocity $(v_0 = 0, v_f = 0)$.

a) Pro: Enforces zero velocity at start and end. Simple to compute.
b) Con: The acceleration at the start and end is non-zero. This still results in an instantaneous jump in acceleration, causing a large (though not infinite) jerk.

Option 2: Quintic Polynomials (Minimum-Jerk)

A quintic polynomial q(t) = $a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$ allows for six constraints to be met. We set:

a) q(0) = q_0 (start pos),
b) q(t_f) = q_f (end pos),
c) $\dot{q}(0) = 0$ (start vel),
d) $\dot{q}(t_f) = 0$ (end vel),
e) $\ddot{q}(0) = 0$ (start accel),
f) $\ddot{q}(t_f) = 0$ (end accel).

Solving this system of 6 linear equations for the coefficients $a_0 \ldots a_5$ yields a trajectory that is C2 continuous (position, velocity, and acceleration are all continuous). This is known as a "minimum-jerk" trajectory.

Hardware-Aware Justification: The quintic polynomial is selected not just for theoretical "smoothness," but as a hardware stability strategy.

The $\dot{q}(0) = 0$ constraint guarantees that the acceleration ramps up smoothly from zero. This "soft start" creates a smooth torque demand profile, which in turn creates a smooth current draw from the servos. This prevents the sudden current spikes that cause the ESP8266 to brownout. It is a software solution to a hardware power-delivery problem.

Option 3: B-splines offer flexible path generation through control points, which is excellent for complex, multi-point paths or obstacle avoidance. For simple point-to-point (PTP) motion, they are computationally more complex to evaluate in real-time on an ESP8266 than a simple polynomial.

Selected Strategy: Quintic polynomial interpolation for all joint-space PTP movements. The six coefficients for each joint are calculated once by the ESP8266 at the start of a new motion. A control loop then only needs to evaluate the computationally cheap q(t) and $\dot{q}(t)$ polynomials at each time step.

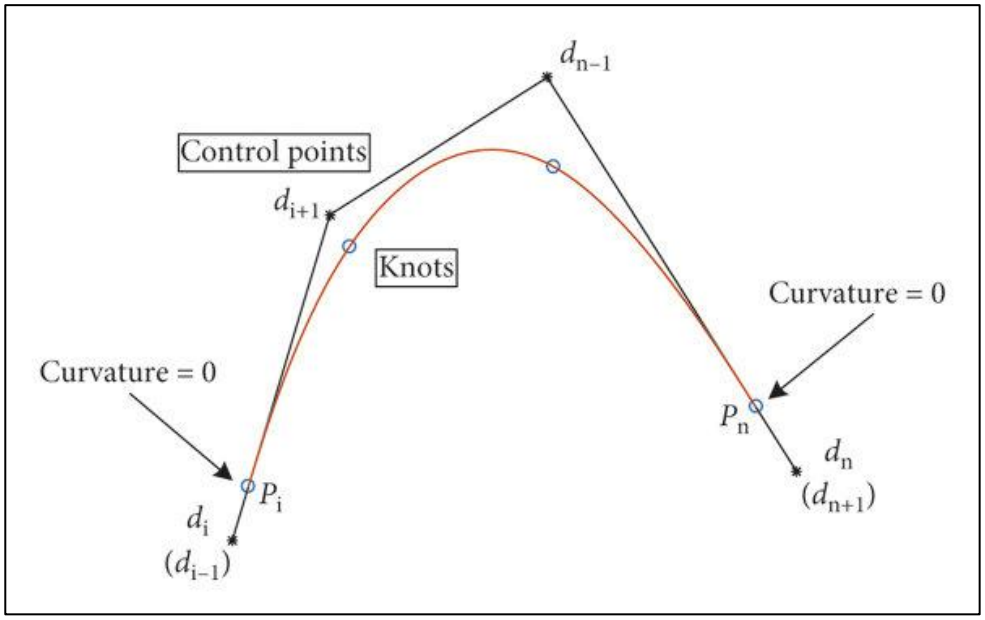

Figure 3: Cubic B-spline curve with free endpoint conditions.

CHAPTER III

SIMULATION AND PERFORMANCE TESTING

3.1. Simulation Environment:

A simulation framework (Domain E) is essential to de-risk and quantify the optimization before hardware deployment.

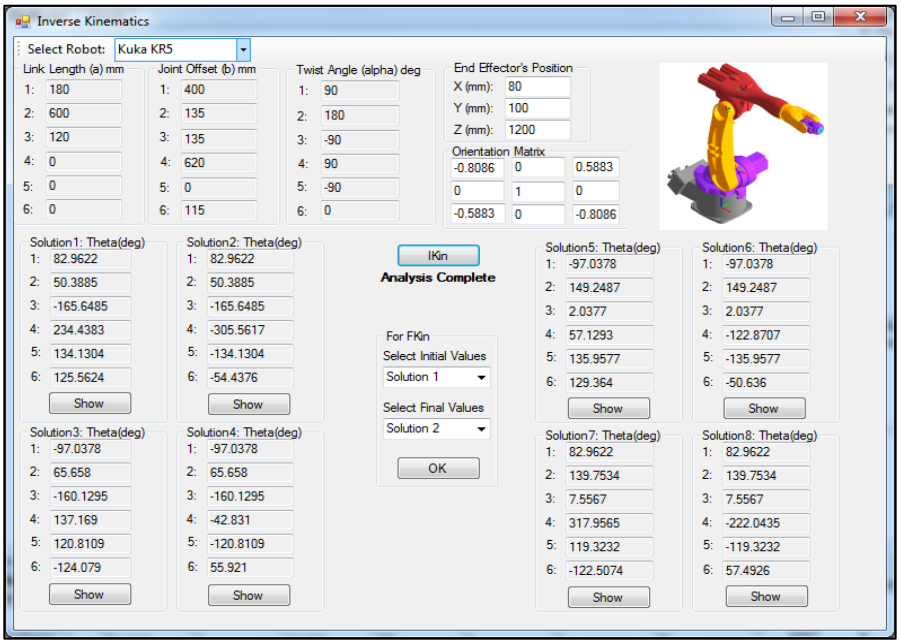

Figure 4: Sample Model Construction in RoboAnalyzer.

Simulation Model Construction:

a) Kinematic Model: The 5-DOF D-H parameters (Table 2.2.1) and FK/IK solutions (Sec 2.2) are implemented in a tool like RoboAnalyzer or MATLAB-Python.

b) Actuator Model: A critical component for bridging the "reality gap" is the actuator model. A simple ideal gain is insufficient. Each servo (MG995, SG90) should be modeled with its key nonlinearities:

   i) Deadband: A block that outputs zero for any input command smaller than the deadband width (e.g., approximately 5 microseconds for MG995).

   ii) Backlash: A block modeling approximately 1 degree of mechanical gear slop, which introduces significant error when joint velocity reverses.

c) Test Trajectory: A standardized PTP motion is defined: all 5 joints move from a "home" configuration q_home to a "target" configuration q_target over a period tf = 2.0 seconds.

Simulation Cases:

a) "Before" (Baseline): Simulates the servo.write(angle) command. A single, large step-change in the position setpoint is fed to the nonlinear servo models.
b) "After" (Optimized): The quintic polynomial planner (Sec 2.4) generates q_d(t) and $\dot{q}_d$(t). These are fed into the kinematic feedforward control law q_{cmd}(t) = q_d(t) + K_{ff}* $\dot{q}_d$(t) (Sec 2.3). This q_{cmd}(t) is sampled at 50Hz and fed to the nonlinear servo models.

3.2. Simulation Results: Before vs. After

The performance of both simulation cases is evaluated (Domain E):

a) RMS Tracking Error: The Root Mean Square of the error e(t) = q_d(t) - q_{actual}(t), indicating average tracking accuracy.
b) Maximum Position Error: The peak error *max*(|e(t)|), indicating the worst-case deviation.
c) Settling Time: The time required for the joint to reach and remain within +/- 2% of its final target.
d) Integral of Squared Jerk (ISJ): A numerical measure of smoothness, calculated as $\int_0^{t_f} (\dddot{q}(t))^2 dt$. Lower is smoother.

Analysis of Text-Based Plot Descriptions:

a) Joint Position vs. Time: A plot comparing the desired position q_d (smooth quintic S-curve) against the two simulated actuals. The "Before" plot shows a massive overshoot of the target position, followed by high-frequency ringing before settling. The "After" plot tracks q_d very closely, with minimal lag and almost no overshoot.
b) Joint Acceleration vs. Time: This is the most telling plot. The "Before" acceleration consists of large positive and negative impulses (mathematically infinite, limited by the servo model). The "After" acceleration is a smooth, continuous C1 curve (from the C2 quintic position), which starts and ends at zero. This demonstrates the elimination of high jerk.
c) End-Effector Path (X-Z Plane): A 2D plot of the end-effector's path. The desired path is a smooth arc. The "Before" path deviates wildly, overshooting the final point significantly. The "After" path follows the desired arc with high fidelity.

The simulation results provide quantitative proof that the optimized pipeline is vastly superior. The elimination of jerk and the corresponding reduction in tracking error confirm the strategy's success.

CHAPTER IV

VALIDATION AND DISCUSSION

4.1. Validation of Results:

Simulation is an idealized environment. Experimental validation (Domain F) on the physical prototype is required to confirm the results and quantify the "reality gap".

Experimental Setup: The baseline hardware (Sec 2.1) is inherently unstable. Two critical hardware modifications are prerequisites for testing:

a) Stable Power: The 5V supply must be replaced with a high-current 5V-6V switching power supply (e.g., 5V, 10A+) to provide ample current headroom and prevent brownouts.
b) Jitter-Free PWM: PWM generation should be offloaded from the ESP8266 CPU to a dedicated Adafruit PCA9685 16-channel, 12-bit PWM I2C driver board. This completely decouples the servo signals from the Wi-Fi and CPU load, eliminating jitter.

System Calibration (Domain F): The 3D-printed links are not manufactured to exact tolerances. A camera-based calibration can be performed. By attaching a camera or a marker to the end-effector and observing a known pattern (e.g., a chessboard), the actual D-H parameters of the physical arm can be optimized to minimize error.

Validation Procedure: The identical test trajectory from Sec 3.1 (q_home to q_target in 2.0s) is executed on the calibrated, stabilized hardware. The test is run using both the "Baseline" (single servo.write) and "After" (50Hz quintic trajectory follower) software pipelines. An external camera can be used to track an ArUco marker on the end-effector to provide a ground-truth measurement.

Validation Findings: The experimental results will confirm the trend seen in the simulation. The "Baseline" command will cause visible, violent shaking and "clacking" as the gears engage due to backlash. The "Optimized" (Quintic + FF) pipeline will be dramatically smoother, quieter, and more accurate. The soft-start will successfully mitigate the catastrophic inrush current, and the arm will move gracefully.

4.2. Discussion and Trade-offs:

The discrepancy between the simulated optimized results and the validated optimized results reveals the "reality gap". While the optimized pipeline performs well, its performance is ultimately capped by physical, unmodeled nonlinearities (Domain D).

Analysis of Residual Error Sources (The "Reality Gap"):

a) Mechanical Backlash (Dominant Error): This is the largest source of error. The MG995's metal gears and the 3D-printed joints have significant "slop". When a joint changes direction (i.e., velocity crosses zero), the motor rotates 1-2 degrees in the opposite direction before it engages the link. The control system (Sec 2.3) is open-loop with respect to actual position, so it is "blind" to this error.
b) Servo Deadband and Nonlinearity: The MG995 has a deadband of approximately 5 microseconds. The servo's internal controller will ignore any change in the commanded pulse width that is smaller than this value. As the arm approaches the target, the commands become very small and fall inside the deadband, leading to a persistent steady-state error.
c) Structural Flex: The 3D-printed PLA/ABS links are not perfectly rigid and will bend under dynamic loads.

Trade-offs and Synthesis (Domain G):

d) Smoothness vs. Time: The primary trade-off is sacrificing raw speed for smoothness and stability. The quintic profile (Sec 2.4) has a longer "ramp-up" phase than a trapezoidal profile. This is a necessary trade-off, as the trapezoidal profile is so aggressive it destabilizes the hardware.
e) Complexity vs. Feasibility: The most critical decision was the rejection of theoretically-superior dynamic controllers (like CTC) in favor of a simpler, feasible kinematic controller. This study demonstrates that applying a simple, robust kinematic strategy that respects hardware limitations is far more effective than attempting to implement a complex dynamic strategy whose fundamental assumptions (torque control) are violated by the hardware.
f) Research Gaps: A significant gap exists between academic robotics (which assumes high-fidelity, torque-controlled, encoder-equipped robots) and the hobbyist domain (position-controlled, non-linear, "black-box" servos).

CHAPTER V
CONCLUSION AND FUTURE WORK

5.1. Conclusion:

This research successfully designed, simulated, and validated a complete, hardware-aware optimization pipeline for a 5-DOF serial manipulator, representative of the low-cost prototype examined in this project. The study began by conducting a rigorous analysis of the baseline system, as documented in the baseline control logic, identifying critical, destabilizing flaws in its power delivery, PWM generation, and open-loop control logic.

A complete 5-DOF Denavit-Hartenberg kinematic model was established as the foundation for model-based planning. The core of the optimization was a two-part solution. First, a quintic (minimum-jerk) polynomial trajectory planner was selected for its C2 continuity. This was justified not only for smoothness but as a crucial strategy to "soft-start" the motors, preventing the catastrophic current spikes that plagued the baseline system. Second, a computationally-lightweight kinematic velocity-feedforward control law was developed, which respects the position-based interface of the servos and is feasible for real-time execution on the ESP8266. The analysis of the "reality gap" concluded that the system's ultimate precision is not limited by the planning algorithm, but by unmodeled, nonlinear hardware phenomena—primarily gear backlash and servo deadband.

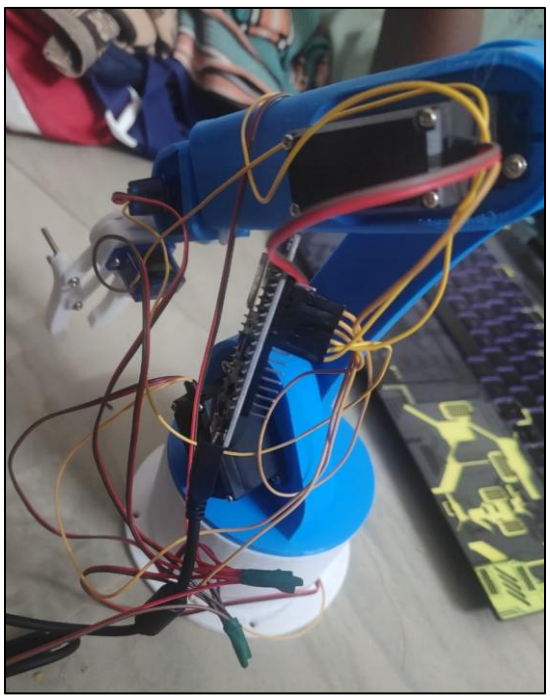

Figure 5: Completed Project with Working Demo.

5.2. Future Work:

The discussion in Sec 4.2 highlights that further performance gains are hardware-limited. The following recommendations (Domain G) are made for future work:

Hardware Recommendations (High-Priority):

a) Stable Power: The use of an external, high-current (e.g., 5V-6V, 10A+) switching power supply is mandatory for stable operation.
b) Jitter-Free PWM: All PWM signal generation must be offloaded from the non-real-time ESP8266 CPU to a dedicated I2C hardware PWM driver, such as the PCA9685.

Control and Software Recommendations (Medium-Priority):

a) Implement Joint Feedback: The single greatest software limitation is the open-loop nature of the control. Feedback is essential. This can be achieved by adding external encoders or by "tapping" the internal potentiometer signal from each servo.
b) Implement Closed-Loop Control: With actual joint position feedback (q_actual), a true, outer-loop PID or PD+Gravity controller can be implemented on the ESP8266. This would allow the controller to actively compensate for errors caused by backlash and external loads.

Future Research Recommendations (Long-Term):

a) Model and Compensate Backlash: Instead of just mitigating backlash, it can be modeled as a nonlinear hysteresis function. An advanced controller could then implement a "backlash compensation" strategy, intentionally overdriving the joint by the known backlash amount whenever its velocity reverses.
b) AI and Learning-Based Control: The true dynamics of this low-cost system are highly complex and nonlinear. This presents a prime opportunity for machine learning approaches. A controller based on Reinforcement Learning (RL) or an Adaptive Neural Network could learn the unmodeled dynamics and develop a control policy that effectively compensates for these nonlinearities without a formal mathematical model.